# SQL Day_01

SQL Fundamentals — Basic to Medium + Practical Queries

**Basic → Medium • Oracle-friendly**

TABLE

ROWS

SQL

DATA

# Day 1 Learning Roadmap



- SQL Fundamentals
- Database → Table → Row → Column
- SELECT
- WHERE + Operators
- NULL Handling
- ORDER BY + FETCH
- Functions
- CASE
- Aggregates
- GROUP BY + HAVING
- Subqueries
- EXISTS / NOT EXISTS
- CTE
- Set Operators

# What is SQL?



- SQL = Structured Query Language.
- Used to create, read, modify and analyze relational data.
- Common database systems: Oracle, PostgreSQL, MySQL, SQL Server.
- SQL is declarative: you describe WHAT data you need; the database decides HOW to retrieve it.
- Today we focus on querying and business analysis.

Think of SQL as asking precise questions to a structured data store.

# Database, Table, Row & Column



- Database: organized collection of related data.
- Table: data arranged as rows and columns.
- Row / record: one entity or transaction.
- Column / attribute: one property of that entity.
- Primary key: column(s) that uniquely identify a row.
- Example: CUSTOMER table → customer_id, name, city, income.

Mental model: Table = Excel sheet; Row = one record; Column = one field.

# Example Data: EMPLOYEES



| EMP_ID | NAME | DEPT | SALARY | CITY |
|---|---|---|---|---|
| 101 | Amit | IT | 70000 | Pune |
| 102 | Neha | HR | 55000 | Mumbai |
| 103 | Ravi | IT | 85000 | Pune |
| 104 | Sara | SALES | 62000 | Nashik |
| 105 | John | IT | NULL | Mumbai |

- EMP_ID → unique identifier
- SALARY for employee 105 is NULL (unknown / missing), not zero.
- This table will be reused in many examples.

# SELECT — Read Data



- SELECT chooses columns from a table.
- SELECT * returns all columns (useful for exploration, but avoid blindly using it in production).
- Select only the columns required by the business question.

```
SELECT name, dept, salary
FROM employees;
```

# SELECT: Expressions & Aliases



```
SELECT
    name,
    salary,
    salary * 12 AS annual_salary,
    city AS location
FROM employees;
```

- Expression: salary * 12 calculates a value.
- AS gives a readable output name (alias).
- Aliases improve reports and downstream analysis.

# WHERE — Filter Rows



- WHERE filters rows before aggregation.
- Use comparison operators: =, <>, !=, >, <, >=, <=.
- Text values normally use single quotes.

```
SELECT name, salary
FROM employees
WHERE dept = 'IT';

SELECT name, salary
FROM employees
WHERE salary >= 70000;
```

# WHERE — AND / OR / NOT



```
-- AND: both conditions must be true
SELECT * FROM employees
WHERE dept = 'IT' AND salary >= 80000;

-- OR: at least one condition is true
SELECT * FROM employees
WHERE city = 'Pune' OR city = 'Mumbai';

-- NOT: reverses a condition
SELECT * FROM employees
WHERE NOT dept = 'HR';
```

💡 Use parentheses when mixing AND and OR so your intent is explicit.

# WHERE — IN, BETWEEN, LIKE



```
-- IN
SELECT * FROM employees
WHERE city IN ('Pune', 'Mumbai');

-- BETWEEN (inclusive)
SELECT * FROM employees
WHERE salary BETWEEN 60000 AND 85000;

-- LIKE pattern matching
SELECT * FROM employees
WHERE name LIKE 'A%';
```

% = any number of characters; _ = exactly one character.

# NULL Handling



- NULL means missing, unknown or not applicable.
- NULL is not equal to 0, an empty string, or FALSE.
- Do not write salary = NULL.
- Use IS NULL / IS NOT NULL.

```
SELECT * FROM employees
WHERE salary IS NULL;

SELECT * FROM employees
WHERE salary IS NOT NULL;
```

# NULL Handling with NVL / COALESCE



```
-- Oracle
SELECT name,
       NVL(salary, 0) AS salary_value
FROM employees;

-- Standard SQL / many databases
SELECT name,
       COALESCE(salary, 0) AS salary_value
FROM employees;
```

- NVL(a,b) is common in Oracle.
- COALESCE(a,b,c,...) returns the first non-NULL value.
- Be careful: replacing NULL with 0 changes the business meaning.

# ORDER BY — Sort Results



```
-- Lowest to highest
SELECT name, salary
FROM employees
ORDER BY salary ASC;

-- Highest to lowest
SELECT name, salary
FROM employees
ORDER BY salary DESC;

-- Multiple sort keys
SELECT name, dept, salary
FROM employees
ORDER BY dept ASC, salary DESC;
```

💡 ASC is default. NULL ordering can vary by database; Oracle supports NULLS FIRST / NULLS LAST.

# LIMIT vs Oracle FETCH FIRST



- MySQL/PostgreSQL commonly use LIMIT.
- Oracle 12c+ supports FETCH FIRST / OFFSET for row limiting.
- Since this deck is Oracle-friendly, use FETCH FIRST in SQL Developer.

```
-- Oracle
SELECT name, salary
FROM employees
ORDER BY salary DESC
FETCH FIRST 3 ROWS ONLY;

-- PostgreSQL / MySQL style
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

# String Functions



```
SELECT
    UPPER(name) AS upper_name,
    LOWER(city) AS lower_city,
    LENGTH(name) AS name_length,
    SUBSTR(name, 1, 3) AS short_name
FROM employees;
```

- UPPER / LOWER → change letter case.
- LENGTH → count characters.
- SUBSTR → extract part of a string.
- Functions are useful for cleaning, matching and reporting.

# Numeric Functions



```
SELECT
    salary,
    ROUND(salary / 1000, 1) AS salary_k,
    TRUNC(salary / 1000) AS salary_k_trunc,
    ABS(salary - 70000) AS distance
FROM employees;
```

- ROUND → rounds a number.
- TRUNC → removes decimal portion in Oracle.
- ABS → absolute difference.
- Always verify function names for your database dialect.

# Date Functions — Oracle Basics



```
SELECT
    SYSDATE AS today,
    ADD_MONTHS(SYSDATE, 1) AS next_month,
    EXTRACT(YEAR FROM SYSDATE) AS year_value
FROM dual;
```

- SYSDATE → current database server date/time in Oracle.
- ADD_MONTHS → shift a date by months.
- EXTRACT → get year/month/day components.
- Date functions are essential in time-based business analysis.

# CASE Expression — Conditional Logic



```
SELECT name, salary,
       CASE
         WHEN salary >= 80000 THEN 'High'
         WHEN salary >= 60000 THEN 'Medium'
         WHEN salary IS NULL THEN 'Unknown'
         ELSE 'Low'
       END AS salary_band
FROM employees;
```

CASE is SQL's IF/ELSE-style expression and is heavily used in reporting.

# CASE — Business Example



- Question: classify customers by spending.
- Business rules can be translated directly into SQL.
- Always define boundary conditions clearly: >= vs >.

```
SELECT customer_id, total_spend,
       CASE
         WHEN total_spend >= 100000 THEN 'VIP'
         WHEN total_spend >= 50000  THEN 'PREMIUM'
         WHEN total_spend >= 10000  THEN 'REGULAR'
         ELSE 'LOW'
       END AS customer_segment
FROM customers;
```

# Aggregate Functions



- COUNT() → count rows / non-NULL values depending on expression.
- SUM() → total.
- AVG() → average of non-NULL values.
- MIN() / MAX() → smallest / largest.
- COUNT(*) counts rows; COUNT(column) ignores NULL values in that column.

```
SELECT
    COUNT(*) AS employees,
    COUNT(salary) AS salary_known,
    SUM(salary) AS total_salary,
    AVG(salary) AS avg_salary,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary
FROM employees;
```

# GROUP BY — Summarize by Category



- GROUP BY creates one result group for each distinct category.
- Every selected non-aggregate column generally must appear in GROUP BY.
- Aggregates calculate a value per group.

```
SELECT dept,
       COUNT(*) AS employee_count,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY dept;
```

# HAVING — Filter Groups



- WHERE filters individual rows.
- HAVING filters groups after aggregation.
- Use HAVING when the condition depends on COUNT, SUM, AVG, etc.

```
SELECT dept,
       COUNT(*) AS employee_count,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY dept
HAVING COUNT(*) >= 2;
```

Shortcut: WHERE = row filter; HAVING = group filter.

# WHERE vs HAVING



```
SELECT dept, AVG(salary) AS avg_salary
FROM employees
WHERE city <> 'Mumbai'        -- filter rows first
GROUP BY dept
HAVING AVG(salary) > 60000;   -- filter groups
```

- WHERE reduces input rows before grouping.
- GROUP BY forms groups.
- HAVING removes groups based on aggregate conditions.
- This distinction is a frequent interview question.

# Subquery — Query Inside a Query



- A subquery is a SELECT nested inside another SQL statement.
- It can return a scalar value, a list, or a table-like result.
- Use it when one query's result is needed by another query.

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

# Subquery with IN



```
SELECT name, dept
FROM employees
WHERE dept IN (
    SELECT dept
    FROM employees
    GROUP BY dept
    HAVING AVG(salary) > 70000
);
```

- Inner query identifies departments whose average salary exceeds 70,000.
- Outer query returns employees belonging to those departments.

# EXISTS — Test for Matching Rows



- EXISTS returns TRUE when the subquery finds at least one matching row.
- It is useful when you care about existence, not the actual returned values.
- Correlate the subquery to the outer row using a key.

```
SELECT c.customer_id, c.name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

# NOT EXISTS — Find Missing Relationships



```
-- Customers who never placed an order
SELECT c.customer_id, c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

- Very useful for 'customers with no orders', 'products never sold', 'employees with no attendance', etc.
- Often safer than NOT IN when NULLs are possible in the subquery.

# CTE — Common Table Expression



- CTE starts with WITH and creates a named temporary result for the query.
- Improves readability and breaks complex logic into steps.
- A CTE exists only for the duration of that SQL statement.

```
WITH dept_stats AS (
    SELECT dept, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept
)
SELECT *
FROM dept_stats
WHERE avg_salary > 60000;
```

# CTE — Multi-Step Business Query



```
WITH customer_sales AS (
    SELECT customer_id, SUM(amount) AS total_sales
    FROM orders
    GROUP BY customer_id
),
ranked AS (
    SELECT customer_id, total_sales,
           RANK() OVER (ORDER BY total_sales DESC) AS rnk
    FROM customer_sales
)
SELECT * FROM ranked WHERE rnk <= 10;
```

This is a bridge from Day 1 SQL fundamentals toward analytical SQL/window functions.

# Set Operators



- UNION → combines results and removes duplicates.
- UNION ALL → combines results and keeps duplicates; usually requires less duplicate-removal work.
- INTERSECT → rows common to both queries.
- MINUS (Oracle) / EXCEPT (many databases) → rows in first query but not second.
- Queries must have compatible column counts and data types.

# UNION vs UNION ALL — Example



```
SELECT city FROM employees
WHERE dept = 'IT'
UNION
SELECT city FROM employees
WHERE dept = 'HR';

SELECT city FROM employees
WHERE dept = 'IT'
UNION ALL
SELECT city FROM employees
WHERE dept = 'HR';
```

Use UNION when duplicates should disappear; UNION ALL when duplicates are meaningful.

# SQL Logical Execution Order

CORE CONCEPT

- 1. FROM / JOIN
- 2. WHERE
- 3. GROUP BY
- 4. HAVING
- 5. SELECT
- 6. DISTINCT
- 7. ORDER BY
- 8. FETCH / LIMIT

The order you WRITE SQL is not the same as the logical order used to reason about the result.

# Business Problem 1 — Top 3 Salaries



- Requirement: return employee name and salary for the three highest salaries.
- Sort descending, then limit the result.

```
SELECT name, salary
FROM employees
WHERE salary IS NOT NULL
ORDER BY salary DESC
FETCH FIRST 3 ROWS ONLY;
```

# Business Problem 2 — Department Metrics

PRACTICE

- Requirement: show department, employee count and average salary.
- Only include departments with at least 2 employees.

```
SELECT dept,
       COUNT(*) AS employee_count,
       ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY dept
HAVING COUNT(*) >= 2
ORDER BY avg_salary DESC;
```

# Business Problem 3 — Customer Activity

PRACTICE

- Requirement: identify customers who have placed at least one order.
- EXISTS is a natural solution because the question is about existence.

```
SELECT c.customer_id, c.name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

# Interview Questions — Day 1



- 1. Difference between WHERE and HAVING?
- 2. Why can't we use = NULL?
- 3. COUNT(*) vs COUNT(column)?
- 4. UNION vs UNION ALL?
- 5. What is a correlated subquery?
- 6. EXISTS vs IN — when can NULLs matter?
- 7. What is a CTE and why use it?
- 8. Explain SQL logical execution order.
- 9. Write a query for the second-highest salary.
- 10. Find departments whose average salary is above the company average.

# Day 1 Practice Set

HANDS-ON

- 1. Find all employees from Pune.
- 2. Find employees with salary between 50,000 and 80,000.
- 3. Show names in uppercase.
- 4. Classify salaries as High / Medium / Low.
- 5. Count employees by city.
- 6. Find cities having more than 1 employee.
- 7. Find employees earning above average salary.
- 8. Find customers with no orders using NOT EXISTS.
- 9. Build a CTE for department salary statistics.
- 10. Combine two city lists using UNION and UNION ALL.

# Day 1 Cheat Sheet



- SELECT → choose columns
- WHERE → filter rows
- CASE → conditional logic
- COUNT / SUM / AVG / MIN / MAX → aggregate
- GROUP BY → create groups
- HAVING → filter groups
- Subquery → query inside query
- EXISTS / NOT EXISTS → relationship existence
- WITH → CTE / readable multi-step query
- UNION / UNION ALL → combine result sets
- ORDER BY → sort; FETCH FIRST → limit rows in Oracle
- NULL → use IS NULL / IS NOT NULL

# Next Step



- Practice the 10 business problems without looking at the answers.
- Then move to JOINs: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN.
- After JOINs: window functions, ranking, date analysis and advanced business SQL.
- Goal: convert a business requirement → SQL logic → query → validated result.

💡 Recommended study cycle: Learn concept → write query → change the data → predict result → execute → explain.